WaferAI · Fab AI Inspector

추가 진행 보고서

10/01 발표 이후 진행한 내용

Phase 5 배포 구성 · Phase 6 재학습 루프와 3분할 · 3분할 재학습 · 운영 모델 교체 완료

항목	내용
팀	팀 Wafer-Eye · 조원곤, 변지민
기준 시점	10/01 5주차 발표 (Phase 0 – 4, 테스트 126건)
보고 범위	10/02 – 10/15 · 3분할 재학습과 운영 모델 교체, 판정 로직 재측정까지
작성일	2026년 10월 13일 (10월 15일 판정 로직 측정 추가)
핵심 변화	처음으로 학습에 쓰지 않은 테스트 셋 성적 확보 — 정확도 97.74% · macro-F1 0.891 (실제 분포 17,504건). 테스트 126 → 190건 · API 18 → 24개 · 설계 API 9개 모두 구현

1. 한눈에 보기

가장 큰 변화 — 오염되지 않은 테스트 성적

발표의 96.45% 는 학습과 섞인 검증 정확도였다. 로트 단위 3분할로 새로 학습한 모델을 학습에 전혀 쓰지 않은 테스트 셋으로 재니 실제 분포 17,504건에서 정확도 97.74%, macro-F1 0.891, 클래스 균형 표본 4,844건에서 정확도 96.24%, macro-F1 0.918 이 나왔다 (4장).

1.1 산출물 8단계 — 10/01 발표와 지금

10/01 발표 3번 슬라이드의 "현재 수치" 열을 그대로 가져와 지금 상태와 나란히 놓았다.

단계	10/01 발표	지금 (10/13)
01 개발 환경	완료 · Docker 환경은 08단계로	Docker 구성 작성 — API·웹 이미지 2종, Compose 3컨테이너. 이미지 빌드 자체는 서버에서 확인
02 DB 구축	테이블 5개	테이블 6개 — retrain_job 추가, ai_model 에 학습 분할·테스트 성적 열 추가. SQL DDL 문서 갱신
03 백엔드	API 18개 · 설계 9개 중 8개	API 24개 · 설계 9개 중 9개 — 남았던 재학습 트리거 구현
04 프론트엔드	설계 화면 8개 중 6개	8개 중 7개 — 화면 5(AI 모델 관리 상단) 추가. 단, 30일 성능 추이 그래프는 아직. 화면 8(계정 설정)은 미구현
05 시스템 통합	시나리오 11개 중 8개	재학습 흐름 추가 확인 — 요청 → 데이터 내려받기 → 학습 → 승격 → 완료 처리를 끝까지 실행. 시나리오 표 수치는 원본 목록과 대조 후 갱신 필요
06 테스트	126건 통과	190건 통과 — SQLite 와 PostgreSQL 16 양쪽에서 전체 통과
07 디버깅	버그 7 · 문서 불일치 6	9건 추가 수정 (5장)
08 CI/CD	예정 (서버 확보 후)	배포 구성·절차서 완료 — GitHub Actions 미착수, 서버 미정

1.2 "다음 발표(10/08)까지 목표" 대비

발표 수정본 8번 슬라이드에 적은 세 가지 목표와 실제 결과다.

목표	결과	내용
시스템 통합 & 테스트 착수	진행	테스트 190건. 처음으로 운영 DB(PostgreSQL 16)에서도 전체를 돌려 SQLite 에서는 드러나지 않던 버그 1건을 찾았다. 시스템통합현황 문서 갱신은 아직
Sensor AI 화면 착수	미착수	Vision 을 서비스 수준으로 먼저 끝내는 쪽을 택했다. 범위를 다시 정해야 한다
CI/CD 준비 & 서버 배정 대응	부분	Docker Compose(Vision + Nginx)와 배포 절차서는 완료해 서버가 생기면 바로 올릴 수 있다. GitHub Actions 초안은 아직이고, 저장소에 GitHub 원격도 아직 없다

2. Phase 5 — 배포 구성 (10/12)

서버 한 대에 Docker Compose 로 올리는 구성을 만들었다. 서버가 아직 없으므로 서버 없이 확인할 수 있는 것은 모두 실제로 돌려 확인했고, 서버에서만 확인할 수 있는 것은 따로 적었다.

2.1 구성

컨테이너	역할	설계상 결정
web (Nginx)	React 정적 파일 제공, /api 를 백엔드로 전달	외부에 열린 유일한 컨테이너. 보안 헤더와 CSP, 업로드 10MB 제한(앱 한도 8MB보다 크게 두어 앱이 이유를 담아 거절하게 함)
api (FastAPI)	추론 · Grad-CAM · 인증 · 재학습 관리	CPU 전용 PyTorch 로 이미지 크기를 줄임. root 가 아닌 계정으로 실행. 모델은 이미지에 굽지 않고 볼륨으로 연결. 기동 시 마이그레이션과 모델 등록 자동
db (PostgreSQL 16)	운영 DB	외부 포트 없음. 데이터는 볼륨에 보존, 백업 스크립트 제공

- 배포 절차서(deploy/README.md): 서버 요구사항 · 처음 배포 · HTTPS(Let's Encrypt) · 백업과 복원 · 재학습으로 모델 바꾸기 · 업데이트 · 문제 해결
- 의존성 분리: 서버용 requirements.txt / 개발·테스트용 requirements-dev.txt
- 빌드에 들어가는 파일을 허용 목록으로 제한 — .env, 가중치, 데이터가 이미지에 섞이지 않음(테스트로 고정)

2.2 Docker 없이 확인한 것

- 서버용 의존성만 깐 빈 환경에서 기동 · 예측 · 히트맵까지 동작
- PostgreSQL 16 을 띄워 전체 테스트 통과, 마이그레이션 올리기·되돌리기 왕복
- 실제 진입점 스크립트로 운영 모드 기동 — 마이그레이션 자동 적용, 모델 등록, 쿠키 Secure·HttpOnly
- Nginx 설정 문법 검사, CSP: Nginx 와 같은 헤더를 붙인 대역 서버로 전 화면 확인 — 위반 0건

2.3 서버에서 처음 확인할 것

이미지 빌드, 컨테이너 사이 네트워크와 볼륨 권한, 실제 Nginx 동작, HTTPS 인증서 발급, 메모리 사용량. 그리고 ONNX Runtime 과 PyTorch 중 그 서버에서 빠른 쪽을 벤치마크 스크립트로 재서 고른다(발표 Q7).

3. Phase 6 — 재학습 루프와 3분할 (10/13)

> 왜 필요했나
>
> 발표의 96.45%는 검증 정확도다. 1학기 학습이 결함 샘플을 학습과 검증에 모두 써서 독립된 테스트 셋이 없었고, 발표에서도 "부분 해결 — 3분할 재학습으로 마무리"라고 했다(11번 슬라이드 4번, 예상 질문 Q1). 또 설계 API 9개 중 마지막 하나인 재학습 트리거가 남아 있었다.

3.1 학습 · 검증 · 테스트 3분할

같은 로트(lot)의 웨이퍼는 결함 모양이 비슷하다. 한 로트가 학습과 테스트에 나뉘어 들어가면 모델이 사실상 본 것을 시험하게 되어 점수가 부풀려진다. 그래서 로트 단위로 묶어 클래스 비율을 유지하며 80 / 10 / 10 으로 나눴다(StratifiedGroupKFold). 로트 10,762개, 분할 사이 겹침 0건. 분할 전체에 지문(split_id)을 붙여, 다른 분할로 학습한 모델과 섞어 비교하는 일을 막는다.

분할	전체	학습에 쓰는 수	쓰임
train	138,137	40,954	학습. 1학기와 같이 정상(none)을 줄여 클래스 균형을 맞춤
val	17,309	5,240	에폭마다 최고 모델 선택
test	17,504	17,504 (전부)	학습에 전혀 쓰지 않음. 실제 분포 그대로와 균형 표본 두 방식으로 잼

split_id 6685ab33cec9096d

3.2 재학습 루프

#	단계	상태	방법
1	검토 정답 500건 누적(웨이퍼 기준) 또는 관리자 수동 요청	PENDING	자동 생성 / 모델 관리 화면
2	관리자가 학습 데이터 내려받기 — 그 시점까지의 검토로 고정	RUNNING	모델 관리 화면
3	GPU 에서 학습 (Colab 노트북 제공)	RUNNING	scripts/train.py --reviews
4	승격 판정 — 고정 테스트 셋으로 직접 재서 기준을 넘을 때만 교체	RUNNING	scripts/promote_model.py
5	models/ 를 서버로 옮기고 api 재시작 → 새 모델 등록, 카운터 초기화	COMPLETED	자동
–	판정 거부 · 학습 실패	FAILED	관리자가 사유와 함께 표시

3.3 승격 기준과 안전장치

- 공정한 비교: 기준 모델이 같은 분할로 학습됐으면, 균형·실제 분포 양쪽 macro-F1 이 0.005 넘게 떨어질 때 거부한다.
- 첫 재학습: 지금 운영 모델(1학기)은 이 테스트 셋의 샘플을 학습에 썼으므로 비교 기준이 될 수 없다. 이때는 균형 macro-F1 하한 0.85 만 보고, 1학기 수치는 참고로만 표시한다.
- 오염 방지: 다른 분할로 학습한 가중치에서 이어 학습하는 것을 학습 스크립트가 거부한다. 그래서 이번 재학습은 1학기 모델이 아니라 ImageNet 가중치에서 시작한다.

- ONNX 불일치 감지: .pth 만 바꾸고 ONNX 를 다시 만들지 않으면 서버가 옛 모델을 조용히 낸다. ONNX 에 원본 해시를 심고 기동 시 대조해, 다르면 PyTorch 로 대신 실행한다.
- 되돌리기: 교체할 때 이전 파일과 판정 기록을 models/archive/ 에 남긴다.

3.4 화면 — AI 모델 관리 (UIUX 화면 5)

책임 엔지니어 이상이 보고, 재학습 요청·데이터 내려받기·실패 처리는 관리자만 할 수 있다.

UIUX 화면 5 명세	구현
모델 성능 카드 (Accuracy + F1, 마지막 업데이트, 학습 데이터 건수)	구현 — 운영 모델, 검증 정확도, 테스트 정확도·macro-F1, 배포일, 학습 분할
30일 성능 추이 그래프 (드리프트 감지)	미구현 — 운영 데이터가 쌓여야 의미가 있음
(명세 외) 재학습 진행 현황	구현 — 500건까지 남은 수, 작업 단계 안내, 작업 목록, 배포 이력

3.5 새 API 6개

메서드	경로	기능	권한
GET	/api/models	모델 목록 · 배포 이력	책임 이상
GET	/api/models/retrain	재학습까지 남은 수 · 진행 중 작업	책임 이상
GET	/api/models/retrain/jobs	재학습 작업 목록	책임 이상
POST	/api/models/retrain	수동 재학습 요청 (설계 API)	관리자
GET	/api/models/retrain/jobs/{id}/dataset	학습 데이터 내려받기 → 학습 중	관리자
POST	/api/models/retrain/jobs/{id}/fail	실패 처리 · 사유 기록	관리자

4. 3분할 재학습 실행 (완료)

이 맥(M3, 메모리 8GB)에서는 학습 중 메모리가 8GB 를 넘어 스왑으로 장비가 멈췄다. 그래서 Colab GPU 에서 돌린다. 학습 코드와 고정 분할 데이터를 묶음 하나로 만들어 Drive 에 올리고, 노트북이 SHA-256 으로 원본과 같은지 확인한 뒤에만 학습으로 넘어가게 했다.

항목	값
장비	Google Colab · Tesla T4 (15GB) · PyTorch 2.11 · 혼합 정밀도(AMP)
시작 가중치	ImageNet (timm coatnet_0_rw_224.sw_in1k)
하이퍼파라미터	1학기와 동일 — 배치 32, AdamW 5e-5, Focal Loss, 최대 15 에폭
속도	초당 141 – 149장 · 에폭당 약 5.2분 · 15 에폭 약 78분 (조기 종료 없이 끝까지)
묶음 확인	조각 3개 이어 붙임, SHA-256 d15afe450c52… 일치 · split_id 일치

4.1 에폭별 검증 정확도

에폭	검증 정확도	걸린 시간	비고
1	92.14%	5.2분	최고 기록 저장
2	93.45%	5.2분	최고 기록 저장
3	95.10%	5.1분	최고 기록 저장
4	94.81%	5.2분	
5	95.48%	5.2분	최고 기록 저장
6	96.15%	5.2분	최고 기록 저장
7	95.80%	5.2분	
8	96.28%	5.2분	최고 기록 저장
9	96.47%	5.2분	최고 기록 저장
10	96.62%	5.2분	최고 기록 저장
11	96.76%	5.2분	최고 기록 저장
12	96.81%	5.2분	최고 기록 저장
13	96.77%	5.2분	
14	96.85%	5.2분	최고 기록 저장
15	96.83%	5.2분	

이 숫자를 1학기 96.45% 와 바로 비교하면 안 된다

1학기 검증 셋은 학습 데이터와 로트가 섞인 무작위 분할이었다. 새 검증 셋은 로트가 학습과 겹치지 않아 더 엄격하므로 낮게 나오는 것이 정상이다. 최종 판단은 학습에 전혀 쓰지 않은 테스트 셋 결과로 한다.

4.2 테스트 결과 — 학습에 쓰지 않은 17,504건

최고 검증 에폭(14)의 모델로 잰 값이다. "실제 분포"는 테스트 셋을 그대로, "균형"은 정상(none)을 줄여 클래스 비율을 맞춘 표본이다. 공장에서 들어오는 웨이퍼에 가까운 쪽은 실제 분포다.

보는 방식	표본 수	정확도	macro-F1
실제 분포 (natural)	17,504	97.74%	0.891
균형 (balanced)	4,844	96.24%	0.918

4.3 클래스별 F1

클래스	F1 (실제 분포)	F1 (균형)	비고
Center	0.931	0.961	
Donut	0.839	0.839	테스트 32건 — 한두 건에 크게 흔들림
Edge-Loc	0.848	0.923	약함
Edge-Ring	0.988	0.991	
Loc	0.786	0.871	약함
Near-full	0.929	0.929	테스트 14건 — 한두 건에 크게 흔들림
Random	0.890	0.897	
Scratch	0.820	0.873	약함
none	0.990	0.981	

- 약한 클래스는 Loc(0.786), Scratch(0.820), Donut(0.839), Edge-Loc(0.848)이다. 1학기 검증에서 약했던 Loc · Donut 과 같은 쪽이다.
- 실제 분포에서 F1 이 더 낮은 것은 정상 웨이퍼가 훨씬 많아서다. 결함 클래스의 재현율은 두 방식이 같고, 정상을 결함으로 잘못 본 건이 늘어 정밀도가 떨어진다. Donut 과 Near-full 은 두 방식의 값이 같다 — 정상을 이 두 클래스로 잘못 본 건이 없다는 뜻이다.

4.4 승격 판정 — 통과, 운영 모델 교체

승격 스크립트는 체크포인트에 적힌 값을 믿지 않고, 내려받은 모델로 테스트 셋을 이 맥(MPS)에서 직접 다시 잰다. 그 값이 Colab(CUDA)에서 잰 값과 소수 넷째 자리까지 같았다. 모델 파일이 온전히 옮겨졌고 서빙 전처리가 학습과 같다는 확인이기도 하다.

테스트 셋	새 모델 정확도	새 모델 macro-F1	1학기 모델 (참고)
실제 분포	97.74%	0.891	98.19% · 0.937
균형	96.24%	0.918	98.25% · 0.971

1학기 모델이 더 높게 나온 이유

1학기 모델은 이 테스트 셋의 결함 샘플 대부분을 학습에 썼다. 본 문제를 다시 푼 점수라 부풀려져 있고, 그래서 비교 기준으로 쓰지 않는다. 첫 재학습은 균형 macro-F1 하한 0.85 로만 판정했고 새 모델은 0.918 로 통과했다. 이제부터의 재학습은 새 모델과 같은 테스트 셋으로 공정하게 비교된다.

- 교체 완료: models/ 의 .pth 와 ONNX 를 새 모델로 바꿨다. ONNX 와 PyTorch 의 확률 최대 오차는 1.8e-07. 이전 파일과 판정 기록은 models/archive/20261013-175053/ 에 있어 되돌릴 수 있다.
- 새 모델로 전체 테스트가 그대로 통과한다.

4.5 판정 로직 효과 — 새 모델 · 테스트 셋 (10/15)

발표의 "사람 검토 없이 95% 통과, 오류 1.40%"는 1학기 모델과 학습에 섞인 검증 셋으로 잰 값이었다. 새 모델과 테스트 셋으로 다시 쟀다(scripts/evaluate_decisions.py). 기본 설정은 확신도 0.80 · 고밀도 0.80 · 저밀도 규칙 꺼짐이다.

	실제 분포 17,504건	균형 4,844건
사람 검토 없이 나감	96.42% (그 안에서 정확도 99.23%)	94.65% (98.41%)
그중 틀린 건 (전체 대비)	130건 = 0.74%	73건 = 1.51%
ㄴ 결함을 정상으로	19건 — Edge-Loc 8 · Loc 5 · Scratch 3 · Center 3	19건
ㄴ 결함 유형 틀림 / 정상을 결함으로	40건 / 71건	40건 / 14건
검토 대기 (그중 AI 1순위가 맞은 비율)	3.58% (57.74%)	5.35% (57.92%)

- 발표와 같은 조건(균형 표본)으로 보면 94.65% 통과 · 1.51% 오류로 거의 같다. 이번 값은 학습에 쓰지 않은 데이터에서 잰 것이라 발표 수치를 처음으로 뒷받침한다.
- 검토로 올라온 건에서 AI 가 맞는 비율이 58% 다. 임계값이 사람이 볼 가치가 있는 건을 골라 올리고 있다.

확신도 임계값별 검토 부담과 새는 불량 (실제 분포)

임계값	검토 대기	틀린 무인 통과	결함을 정상으로
0.70	2.14%	1.22%	27건
0.75	2.78%	1.00%	22건
0.80 (기본)	3.58%	0.74%	19건
0.85	5.00%	0.49%	12건
0.90	10.93%	0.17%	3건
0.95	99.92%	0.00%	0건

- 0.80 → 0.85 로 올리면 검토가 1.4%p 늘고 정상으로 새는 결함이 19 → 12건으로 준다. 품질과 업무량 사이의 선택이라 현장 기준으로 정할 일이다.
- 0.92 를 넘기면 사실상 전부 검토로 간다. 이 모델의 확신도는 대부분 0.90 – 0.92 에 몰려 있다(Focal Loss 로 학습한 모델의 특성, 0.92 이상은 1.3%). 설정 화면은 0.99 까지 허용하므로 잘못 올리면 검토 대기열이 넘친다.
- 저밀도 규칙은 여전히 해롭다. 켜면 결함을 정상으로 내보내는 건이 19 → 140건으로 7배가 된다.

• 이미지로 올린 경우도 300건 모두 데이터 파일로 올렸을 때와 예측이 같았다(1학기 모델은 297/300).

4.6 남은 순서

- 서버가 생기면 models/ 를 옮기고 api 를 재시작한다. 기동하며 새 모델이 등록된다.
- 설정 화면의 확신도 상한을 이 모델이 실제로 내는 범위에 맞춘다(지금 0.99).

5. 이 기간에 찾아 고친 문제

#	문제	원인	조치
1	PostgreSQL 에서 settings 기본키에 유니크 제약 이름이 붙음. 그 이름으로 정리하면 기본키가 지워짐	PostgreSQL 이 같은 열의 중복 제약을 하나로 합침. SQLite 에서는 안 드러남	실제 제약 종류를 확인해 정리하는 마이그레이션. 두 DB 에서 왕복 확인
2	관리자 생성 스크립트가 DB 비밀번호가 든 접속 주소를 출력	주소를 그대로 출력	비밀번호를 *** 로 가림
3	.env 가 이미지에 들어가는지 보는 테스트가 항상 통과	저장소에 .env 가 없어 검사 대상이 없었음	파일이 있다고 가정하고 규칙 자체를 검사
4	같은 가중치를 다시 등록하면 활성 모델이 2개	새 모델일 때만 나머지를 비활성화	등록 때마다 나머지를 비활성화 (되돌리기 대비)
5	모델 등록 시 NOT NULL 오류	백엔드 이름을 나중에 채움	생성할 때 함께 지정
6	승격 판정 스크립트 IndexError	빈 목록이 실수 배열이 되어 인덱스로 못 씀	정수형 지정, 회귀 테스트 추가
7	맥에서 학습 중 메모리 8.3GB → 스왑으로 장비 정지	MPS 메모리 상한 없음, 평가 배치 고정 64	상한 설정, 평가 배치를 학습 배치에 맞춤. 전체 학습은 Colab
8	PostgreSQL 전체 테스트가 34분 넘게 안 끝남	서버가 뜰 때마다 113MB ONNX 세션을 3개 만들고, 끝난 테스트 앱이 해제되지 않음	세션 1개로, 종료 시 해제 → 2.4분. 재발 방지 테스트
9	테스트가 DB 연결을 닫지 않음	엔진을 만들고 해제하지 않음	해제 추가. PostgreSQL 에서 다음 테스트의 스키마 삭제를 막을 수 있었음

6. 남은 일

항목	상태	필요한 것
재학습 모델 승격 판정 · 교체	완료	통과 · 교체 완료. 서버 배포 시 models/ 를 옮기면 됨
판정 임계값 효과 재측정	완료	무인 통과 96.42% · 틀린 건 0.74% (4.5절)
설정 화면 확신도 상한	미착수	0.92 이상이면 거의 전부 검토로 감 — 상한·경고 필요
서버 확보 · 배포	미정	학교·학과 지원 확인. 권장 4 vCPU · 8GB · 50GB
ONNX / PyTorch 선택	서버 필요	배포 서버에서 벤치마크

항목	상태	필요한 것
GitHub 원격 · Actions	미착수	원격 저장소를 만들고 push 한 뒤 CI 구성
화면 8 계정 설정 · 사용자 관리	API 완료	화면만 붙이면 됨
화면 5 성능 추이 그래프	미구현	운영 데이터가 쌓인 뒤
시나리오 표 · 버그 리포트 갱신	미착수	이 보고서 내용 반영
Sensor AI	미착수	이번 학기 범위 결정
1학기 설계 문서 정리	미착수	손실 함수, weight_decay, 정확도 표기 등 코드와 다른 부분

부록. 10/01 이후 커밋

커밋	날짜	내용
0f5b03a	10/12	Phase 5: Docker Compose + Nginx 배포 구성
559f7be	10/13	Phase 6: 재학습 루프와 train/val/test 3분할
2135cb2	10/13	DB 스키마 SQL 문서를 최신 마이그레이션 기준으로 갱신
30824c2	10/13	Colab 묶음 9MB 조각 업로드와 해시 확인

테스트 190건 구성: 인증 32 · 배포 구성 30 · 재학습 34 · 서버 23 · 히트맵 API 20 · 판정 12 · 전처리 10 · Grad-CAM 10 · 이미지 어댑터 9 · 백엔드 일치 7 · 마이그레이션 3. 이 중 배포 구성 30건과 재학습 34건이 이번 기간에 추가됐다.